

The Krasnosel'skiĭ-Mann Iterative Method

Chapter 4: Relations to Operator Splitting Methods

Based on Dong, Cho, He, Pardalos & Rassias

Where we are

- **Chapter 2** gave us the vocabulary: nonexpansive / firmly nonexpansive / averaged operators, maximally monotone operators, subdifferentials, resolvents and proximal operators.
- **Chapter 3** gave us *one* algorithm template, the Krasnosel'skiĭ–Mann (KM) iteration

$$x_{n+1} = (1 - \lambda_n)x_n + \lambda_n T x_n, \quad T \text{ averaged (or nonexpansive),}$$

and its convergence theory (weak convergence to a point of $\text{Fix}(T)$ under mild conditions on $\{\lambda_n\}$).

- **Chapter 4's punchline:** gradient descent, the proximal point algorithm, forward-backward splitting, Douglas–Rachford, Davis–Yin, and primal-dual splitting are *not* six different algorithms. They are *one* algorithm – “iterate an averaged operator” – wearing six different operators T .
- Roadmap: 4.1 gradient descent, 4.2 proximal point, 4.3 splitting methods for $0 \in A(x) + B(x)$ (or sums of several operators).

Notation you'll need / recap I

Averaged operator (recap from Ch. 2–3)

$T : \mathcal{H} \rightarrow \mathcal{H}$ is α -averaged ($\alpha \in (0, 1)$) if there is a nonexpansive R with $T = (1 - \alpha)\text{Id} + \alpha R$. Equivalently,

$$\|Tx - Ty\|^2 \leq \|x - y\|^2 - \frac{1 - \alpha}{\alpha} \|(\text{Id} - T)x - (\text{Id} - T)y\|^2, \quad \forall x, y \in \mathcal{H}.$$

Averaged operators are exactly the class of maps for which the KM iteration $x_{n+1} = (1 - \lambda_n)x_n + \lambda_n Tx_n$ converges (weakly) to a fixed point, for suitable $\{\lambda_n\}$. *Firmly nonexpansive* = $\frac{1}{2}$ -averaged.

Notation you'll need / recap II

Resolvent / proximal operator (recap)

For a maximally monotone $A : \mathcal{H} \rightrightarrows \mathcal{H}$ and $\gamma > 0$, the *resolvent* is

$$J_{\gamma A} := (\text{Id} + \gamma A)^{-1}.$$

$J_{\gamma A}$ is single-valued, firmly nonexpansive, and $\text{Fix}(J_{\gamma A}) = \text{zer}(A) := \{x : 0 \in A(x)\}$. When $A = \partial f$ for $f \in \Gamma_0(\mathcal{H})$ (proper, convex, lsc), $J_{\gamma \partial f} = \text{prox}_{\gamma f}$, the *proximal operator* of f . Special case: $f = \delta_C$ (indicator of a closed convex C) gives $\text{prox}_{\gamma \delta_C} = P_C$, the metric projection onto C , for every $\gamma > 0$.

Notation you'll need / recap III

Cocoercive operator (new name, old idea)

$B : \mathcal{H} \rightarrow \mathcal{H}$ is β -cocoercive ($\beta > 0$) if

$$\langle x - y, Bx - By \rangle \geq \beta \|Bx - By\|^2, \quad \forall x, y \in \mathcal{H}.$$

By the Baillon–Haddad theorem, if f is convex with L -Lipschitz gradient then ∇f is $\frac{1}{L}$ -cocoercive – this is exactly what makes an explicit (forward) gradient step behave well.

Notation you'll need / recap IV

The monotone inclusion problem

Given a maximally monotone $A : \mathcal{H} \rightrightarrows \mathcal{H}$,

$$\text{Find } x \in \mathcal{H} \text{ such that } 0 \in A(x). \quad (4.3)$$

Minimizing $f \in \Gamma_0(\mathcal{H})$ is the special case $A = \partial f$ (Fermat's rule): minimizers of f = zeros of ∂f = fixed points of $J_{\gamma\partial f} = \text{prox}_{\gamma f}$.

Why “split” at all?

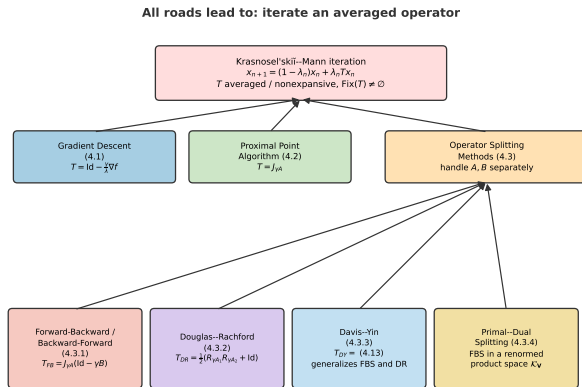
Real problems often look like $0 \in A(x) + B(x)$ (or a sum of several operators). In principle the proximal point algorithm could be applied directly to $A + B$ – but $J_{\gamma(A+B)}$ is almost never computable in closed form, even when $J_{\gamma A}$ and $J_{\gamma B}$ (or a gradient step for B) individually are cheap. *Operator splitting* means: build an iteration that only ever calls $J_{\gamma A}$, $J_{\gamma B}$, or ∇ (smooth part) *separately*, never $J_{\gamma(A+B)}$ jointly. Typical division of labor: one piece is smooth/easy (cocoercive, take a gradient step), the other is nonsmooth/simple-to-prox (maximally monotone, take a resolvent step).

Notation you'll need / recap V

Running example (carried over from earlier chapters)

$C = \{x \in \mathbb{R}^2 : \|x\| \leq 1\}$, the closed unit disk. For $p = (3, 4)$, the projection is $P_C(p) = p/\|p\| = (0.6, 0.8)$. Every algorithm in this chapter will, in some instantiation, reproduce exactly this point.

The big picture: everything is an averaged-operator iteration



4.1 Gradient descent – why, and the formula

Why would you want this?

You want to minimize a *smooth* convex function $f : \mathcal{H} \rightarrow \mathbb{R}$ with L -Lipschitz gradient,

$$\min_{x \in \mathcal{H}} f(x). \quad (4.1)$$

No constraints, no nonsmooth part – the simplest possible instance of “find a zero of a monotone operator” ($A = \nabla f$, single-valued and monotone by convexity). The natural move: walk downhill along $-\nabla f$.

Gradient Descent Algorithm

Let $f : \mathcal{H} \rightarrow \mathbb{R}$ be convex with L -Lipschitz gradient, $\{\gamma_n\} \subset (0, 2/L)$. Choose $x_0 \in \mathcal{H}$ arbitrarily and compute

$$x_{n+1} = x_n - \gamma_n \nabla f(x_n), \quad n \geq 0. \quad (4.2)$$

Symbols: γ_n is the step size; the range $(0, 2/L)$ is exactly what makes this converge.

4.1 Derivation: GD *is* a KM iteration I

Step 1 – Baillon–Haddad

∇f is L -Lipschitz $\iff T := \text{Id} - \frac{2}{L}\nabla f$ is nonexpansive, and $\text{Argmin}_{x \in \mathcal{H}} f(x) = \text{Fix}(T)$. (This is the Hilbert-space upgrade of “convex + Lipschitz gradient $\Rightarrow \nabla f$ is cocoercive with constant $1/L$ ”.)

Step 2 – rewrite the GD step as a KM step

Start from (4.2) and insert a free parameter $\lambda_n \in (0, 1]$:

$$x_{n+1} = x_n - \gamma_n \nabla f(x_n) = (1 - \lambda_n)x_n + \lambda_n \left(x_n - \frac{\gamma_n}{\lambda_n} \nabla f(x_n) \right).$$

This is just algebra: $(1 - \lambda_n)x_n + \lambda_n x_n = x_n$, and the leftover $-\gamma_n \nabla f(x_n)$ is absorbed by writing it as $\lambda_n \cdot \left(-\frac{\gamma_n}{\lambda_n} \nabla f(x_n) \right)$.

4.1 Derivation: GD *is* a KM iteration II

Step 3 – identify T and check it is averaged

Define

$$T := \text{Id} - \frac{\gamma_n}{\lambda_n} \nabla f.$$

Choosing $\lambda_n = \frac{\gamma_n L}{2}$ makes $\frac{\gamma_n}{\lambda_n} = \frac{2}{L}$, so $T = \text{Id} - \frac{2}{L} \nabla f$ is *exactly* the nonexpansive operator from Step 1. Then (4.2) is the KM iteration

$$x_{n+1} = (1 - \lambda_n)x_n + \lambda_n T x_n, \quad \lambda_n = \frac{\gamma_n L}{2},$$

and $\gamma_n \in (0, 2/L) \iff \lambda_n \in (0, 1)$, exactly the admissible KM relaxation range. Also $\text{Fix}(T) = \text{Argmin } f$: the fixed points of T are precisely the minimizers we wanted.

4.1 Derivation: GD *is* a KM iteration III

Takeaway

Gradient descent is nothing but KM iteration of the nonexpansive operator $T = \text{Id} - \frac{2}{L} \nabla f$, with the relaxation parameter λ_n tied to the step size by $\lambda_n = \gamma_n L/2$. Small step size $\Leftrightarrow$ small (safe) relaxation.

4.1 Running example: gradient descent on a quadratic

Setup

$f(x) = \frac{1}{2}\|x - p\|^2$ with $p = (3, 4)$. Then $\nabla f(x) = x - p$ is 1-Lipschitz ($L = 1$), so $\gamma_n \in (0, 2)$ is admissible. Take $\gamma_n \equiv 0.5$, $x_0 = (0, 0)$:

$$x_{n+1} = x_n - 0.5(x_n - p) = 0.5x_n + 0.5p.$$

n	x_n	$\ x_n - p\ $
0	(0.000, 0.000)	5.000
1	(1.500, 2.000)	2.500
2	(2.250, 3.000)	1.250
3	(2.625, 3.500)	0.625
4	(2.8125, 3.750)	0.3125

Unconstrained minimizer

No constraint here, so GD converges to the unconstrained minimizer $p = (3, 4)$ itself – *not* the disk-projection point $(0.6, 0.8)$. We reuse this exact f below (Section 4.3.1) once we add the disk constraint back in

Python: gradient descent as a KM iteration

```
1  import numpy as np
2
3  p = np.array([3.0, 4.0])
4  L = 1.0                                # Lipschitz constant of grad f(x) = x - p
5  grad_f = lambda x: x - p
6
7  def gd_step(x, gamma):
8      return x - gamma * grad_f(x)
9
10 def km_view_step(x, gamma):
11     # same update, written as (1-lam)*x + lam*T(x), T = Id - (2/L) grad f
12     lam = gamma * L / 2.0
13     T = lambda z: z - (2.0 / L) * grad_f(z)
14     return (1 - lam) * x + lam * T(x)
15
16 x = np.array([0.0, 0.0])
17 gamma = 0.5
18 for n in range(5):
19     print(n, x, np.linalg.norm(x - p))
20     x = gd_step(x, gamma)                # identical to km_view_step(x, gamma)
21 # -> converges geometrically to p = (3, 4)
```

4.2 Proximal point algorithm – why, and the setup

Why would you want this?

Gradient descent needs f smooth. Many important problems are not: f may be an indicator function, an ℓ_1 norm, etc. Fermat's rule (Thm 2.1) says minimizing $f \in \Gamma_0(\mathcal{H})$ is equivalent to finding a zero of ∂f . The proximal point algorithm solves the general monotone inclusion (4.3) for *any* maximally monotone A (not just subdifferentials), by repeatedly applying its resolvent.

Proximal Point Algorithm

Let $A : \mathcal{H} \rightrightarrows \mathcal{H}$ be maximally monotone with $\text{zer}(A) \neq \emptyset$, $\{\gamma_n\} \subset (0, +\infty)$, $\{\lambda_n\} \subset [0, 2]$. Choose $x_0 \in \mathcal{H}$ arbitrarily and compute

$$x_{n+1} = (1 - \lambda_n)x_n + \lambda_n J_{\gamma_n A}(x_n), \quad n \geq 0. \quad (4.4)$$

With $\lambda_n \equiv 1$ this is Rockafellar's classical *proximal point algorithm*; the general scheme (4.4) is the *generalized proximal point algorithm* of Eckstein–Bertsekas.

4.2 PPA *is* already a KM iteration, verbatim

Identify T

Take $T := J_{\gamma_n A}$. Since $J_{\gamma_n A}$ is firmly nonexpansive (hence $\frac{1}{2}$ -averaged, in particular averaged), and

$$\text{Fix}(J_{\gamma_n A}) = \text{zer}(A),$$

the scheme (4.4) is *literally* the KM iteration $x_{n+1} = (1 - \lambda_n)x_n + \lambda_n T x_n$ with no algebraic massaging required at all – unlike gradient descent, no re-parametrization step was needed.

Why this matters

Because $T = J_{\gamma_n A}$ is already averaged for every $\gamma_n > 0$ and every maximally monotone A , the KM convergence theory transfers immediately: (4.4) converges weakly to a point of $\text{zer}(A)$ whenever $\{\lambda_n\} \subset [0, 2]$ satisfies the usual divergence condition $\sum_n \lambda_n(2 - \lambda_n) = \infty$.

4.2 Running example: PPA reduces to the earlier projection

Instantiation

Let $A = \partial\delta_C$, the subdifferential of the indicator of the unit disk C (i.e. the normal cone operator N_C). Then, for every $\gamma_n > 0$,

$$J_{\gamma_n A} = \text{prox}_{\gamma_n \delta_C} = P_C$$

– the projection onto C , independent of γ_n ! With $\lambda_n \equiv 1$, (4.4) becomes the utterly familiar

$$x_{n+1} = P_C(x_n).$$

Numbers

Start at $x_0 = p = (3, 4)$ (outside C). Then $x_1 = P_C(x_0) = (3, 4)/5 = (0.6, 0.8) \in C$, and since $P_C(y) = y$ for $y \in C$, $x_n = (0.6, 0.8)$ for every $n \geq 1$: convergence in *one step*. This is literally the running-example projection computation from earlier chapters, now seen as an instance of the proximal point algorithm with $A = \partial\delta_C$.

Python: proximal point algorithm for $A = \partial\delta_C$

```
1 import numpy as np
2
3 def P_C(x):                                #  $J_{\{\gamma A\}}$  for  $A = \text{subdifferential of}$ 
4     n = np.linalg.norm(x)                  # the indicator of the unit disk -- same
5     return x / n if n > 1 else x           # for every  $\gamma > 0$ 
6
7 def ppa_step(x, lam=1.0):
8     return (1 - lam) * x + lam * P_C(x)
9
10 x = np.array([3.0, 4.0])
11 for n in range(3):
12     print(n, x)
13     x = ppa_step(x)
14 # n=0: [3. 4.]
15 # n=1: [0.6 0.8]    <- Fix(P_C) reached: converged in one step
16 # n=2: [0.6 0.8]
```

4.3 The operator splitting methods – setting the stage

Problem 4.1: the general template

Let $B : \mathcal{H} \rightarrow \mathcal{H}$ be β -cocoercive for some $\beta > 0$, $m \geq 1$, and $A_i : \mathcal{H} \rightrightarrows \mathcal{H}$ maximally monotone for $i = 1, \dots, m$. Find

$$x \in \mathcal{H} \text{ such that } 0 \in B(x) + \sum_{i=1}^m A_i(x). \quad (4.5)$$

Why not just run PPA on $A_1 + \dots + A_m + B$?

Even when every $J_{\gamma A_i}$ and ∇ of the smooth piece are individually cheap, the resolvent of the *sum* is essentially never available in closed form. *Operator splitting methods* are iterations that only ever call the resolvents/gradients of the pieces *separately*. The rest of Section 4.3 exhibits four such methods, all of which turn out to be KM iterations for a cleverly built averaged operator T .

4.3.1 Forward-backward splitting – why, and the operator $\mathbf{I}$

Problem 4.1.1 ($m = 1$)

Find $x \in \mathcal{H}$ such that $0 \in (A + B)(x)$, where (a) A is maximally monotone, (b) B is β -cocoercive for some $\beta > 0$, (c) $\text{zer}(A + B) \neq \emptyset$. (4.6)

Why would you want this?

B smooth/cocoercive $\Rightarrow$ an explicit *forward* (gradient-like) step is cheap; A possibly nonsmooth $\Rightarrow$ only its resolvent (an *implicit/backward* step) is cheap. Forward-backward splitting alternates the two: take a forward step to “deal with” B , then a backward step to “deal with” A .

4.3.1 Forward-backward splitting – why, and the operator II

Lemma 4.1 – the averaged operator

Let $\gamma \in (0, 2\beta)$ and define

$$T_{FB} := J_{\gamma A}(\text{Id} - \gamma B).$$

Then (1) T_{FB} is $\frac{2\beta}{4\beta - \gamma}$ -averaged; (2) $\text{zer}(A + B) = \text{Fix}(T_{FB})$.

4.3.1 Forward-backward splitting – why, and the operator III

The forward-backward splitting method

Let $\gamma \in (0, 2\beta)$, $\{\lambda_n\} \subset [0, \frac{4\beta-\gamma}{2\beta}]$ with $\sum_n \lambda_n (\frac{4\beta-\gamma}{2\beta} - \lambda_n) = \infty$. Choose $x_0 \in \mathcal{H}$ and iterate

$$x_{n+1} = (1 - \lambda_n)x_n + \lambda_n T_{FB}x_n, \quad n \geq 0. \quad (4.7)$$

Unwound: $x_{n+1} = (1 - \lambda_n)x_n + \lambda_n J_{\gamma A}(x_n - \gamma B(x_n))$ – an explicit gradient-type step on B , then an implicit resolvent step on A , then a KM relaxation.

4.3.1 Forward-backward splitting – why, and the operator IV

(B) Backward-forward splitting – Lemma 4.2

Swap the order: $T_{BF} := (\text{Id} - \gamma B)J_{\gamma A}$ is also $\frac{2\beta}{4\beta - \gamma}$ -averaged, with $\text{zer}(A_\gamma + B_{-\gamma}) = \text{Fix}(T_{BF})$ (a shifted version of the same zero set). The iteration

$$x_{n+1} = (1 - \lambda_n)x_n + \lambda_n T_{BF} x_n \quad (4.8)$$

does the resolvent step *first*, the gradient step *second*. It arises from time-discretizing the regularized Newton method for maximally monotone operators, and shares the same basic building blocks as FBS, just organized differently.

4.3.1 Running example: FBS on quadratic + disk indicator

Instantiation

$f(x) = \frac{1}{2}\|x - p\|^2$ (smooth, $B = \nabla f$, 1-cocoercive, $p = (3, 4)$), $g = \delta_C$ (nonsmooth indicator of the unit disk, $A = \partial g$, $J_{\gamma A} = P_C$ for all γ). With $\gamma = 0.5 \in (0, 2)$, $\lambda_n \equiv 1$, $x_0 = (-2, 3)$:

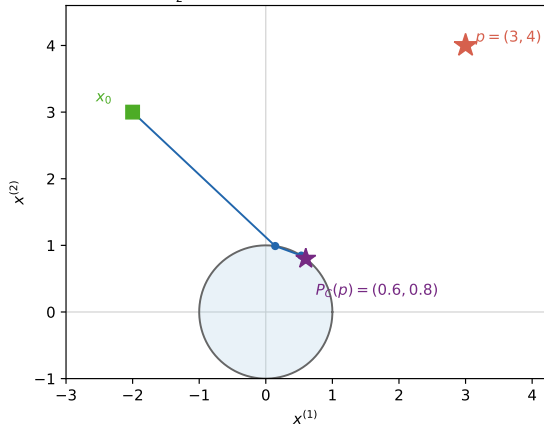
$$x_{n+1} = P_C(x_n - 0.5(x_n - p)).$$

n	x_n	$\ x_n - (0.6, 0.8)\ $
0	(-2.000, 3.000)	3.406
1	(0.141, 0.990)	0.496
2	(0.533, 0.846)	0.082
3	(0.589, 0.808)	0.014
6	(0.5999, 0.80004)	6.3×10^{-5}

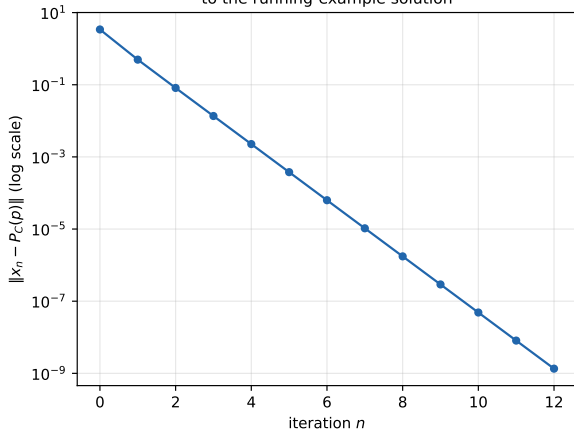
Converges to the *same* point $(0.6, 0.8) = P_C(p)$ as the earlier running example – now obtained by *splitting* $f + \delta_C$ instead of directly projecting.

4.3.1 The iterates, geometrically and in error

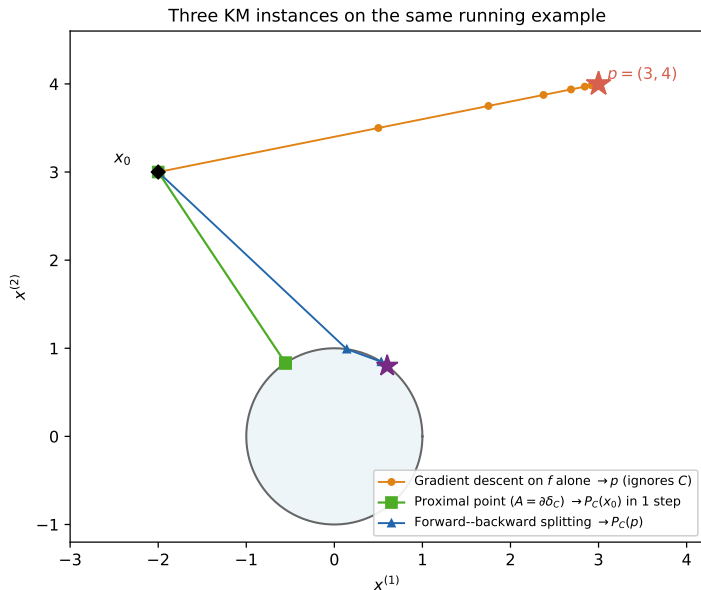
Forward--backward splitting iterates
 $f(x) = \frac{1}{2}\|x - p\|^2$ (smooth), $g = \delta_C$ (nonsmooth)



Linear convergence of FBS
to the running-example solution



4.3.1 Three KM instances, one running example



Python: forward-backward splitting

```
1 import numpy as np
2
3 p = np.array([3.0, 4.0])
4 grad_f = lambda x: x - p           # B = grad f, 1-cocoercive (L = 1)
5
6 def P_C(x):                         # J_{\gamma A}, A = subdifferential of delta_C
7     n = np.linalg.norm(x)
8     return x / n if n > 1 else x
9
10 def fbs_step(x, gamma=0.5):
11     y = x - gamma * grad_f(x)      # forward step on B
12     return P_C(y)                  # backward step on A
13
14 x = np.array([-2.0, 3.0])
15 target = np.array([0.6, 0.8])
16 for n in range(7):
17     print(n, x, np.linalg.norm(x - target))
18     x = fbs_step(x)
19 # -> converges linearly to (0.6, 0.8) = P_C(p)
```

4.3.2 Douglas–Rachford splitting – why, and the operator I

Problem 4.1.2 ($B = 0$, $m = 2$)

Find $x \in \mathcal{H}$ such that $0 \in (A_1 + A_2)(x)$, where A_1, A_2 are maximally monotone and $\text{zer}(A_1 + A_2) \neq \emptyset$.
(4.9)

Why would you want this?

FBS needed one of the two pieces to be *cocoercive* (smooth). Douglas–Rachford drops that requirement entirely: both A_1, A_2 may be nonsmooth maximally monotone operators (e.g. two normal-cone operators for a convex-feasibility problem, or two nonsmooth regularizers). All you need are the two resolvents.

Reflected resolvent (recap)

Write $R_{\gamma A} := 2J_{\gamma A} - \text{Id}$ (the reflection through the resolvent). $R_{\gamma A}$ is nonexpansive whenever A is maximally monotone, because $J_{\gamma A}$ is firmly nonexpansive.

4.3.2 Douglas–Rachford splitting – why, and the operator II

Lemma 4.3 – the averaged operator

Let $\gamma > 0$ and define

$$T_{DR} := \frac{1}{2}(R_{\gamma A_1} R_{\gamma A_2} + \text{Id}).$$

Then: (1) $R_{\gamma A_1} R_{\gamma A_2}$ is nonexpansive; (2) $T_{DR} = J_{\gamma A_1}(2J_{\gamma A_2} - \text{Id}) - J_{\gamma A_2} + \text{Id}$ is firmly nonexpansive (i.e. $\frac{1}{2}$ -averaged – being the average of a nonexpansive map and Id is automatic); (3) $\text{zer}(A_1 + A_2) = J_{\gamma A_2}(\text{Fix}(T_{DR}))$.

4.3.2 Douglas–Rachford splitting – why, and the operator III

The Douglas–Rachford splitting method

Let $\gamma > 0$, $\{\lambda_n\} \subset [0, 2]$ with $\sum_n \lambda_n(2 - \lambda_n) = \infty$. Choose $x_0 \in \mathcal{H}$; for each $n \geq 0$,

$$\begin{cases} y_n = J_{\gamma A_2} x_n, \\ z_n = J_{\gamma A_1} (2y_n - x_n), \\ x_{n+1} = x_n + \lambda_n(z_n - y_n), \end{cases} \quad (4.10)$$

which is exactly the KM iteration $x_{n+1} = (1 - \lambda_n)x_n + \lambda_n T_{DR}(x_n)$. (4.11)

4.3.2 Douglas–Rachford splitting – why, and the operator IV

Remarks

- $\lambda_n \equiv 2$ recovers the *Peaceman–Rachford* splitting method.
- The solution to (4.9) is recovered as $y = J_{\gamma A_2}(x^*)$ where $x^* \in \text{Fix}(T_{DR})$ – note the fixed point of T_{DR} is generally *not itself* the solution, only its image under $J_{\gamma A_2}$ is.
- Douglas–Rachford is, in turn, a special case of the proximal point algorithm (via a product-space trick), and is closely related to ADMM and the primal-dual hybrid gradient method.

4.3.2 Running example (extended): two-disk feasibility

A natural extension of the running example

Let $A_1 = N_{C_1}$, $A_2 = N_{C_2}$ (normal cones = subdifferentials of indicators), with C_1 the unit disk centered at the origin (our familiar C) and C_2 the unit disk centered at $(1, 0)$. Then $J_{\gamma A_i} = P_{C_i}$ for every γ , and $\text{zer}(A_1 + A_2)$ is exactly the lens-shaped intersection $C_1 \cap C_2$ – Douglas–Rachford becomes a *projections onto convex sets* method for convex feasibility.

Numbers ($\gamma = 1$, $\lambda_n \equiv 1$, $x_0 = (2, 2)$)

n	x_n	$y_n = P_{C_2}(x_n)$
0	(2.000, 2.000)	(1.447, 0.894)
1	(1.447, 0.894)	(1.447, 0.894)
2	(0.851, 0.526)	(0.851, 0.526)
≥ 3	(0.851, 0.526)	(0.851, 0.526)

The limit $(0.851, 0.526)$ lies on ∂C_1 and strictly inside C_2 : a genuine point of $C_1 \cap C_2$, found without ever forming $J_{\gamma(A_1+A_2)}$ directly.

Python: Douglas–Rachford splitting

```
1 import numpy as np
2
3 c1, c2 = np.array([0.0, 0.0]), np.array([1.0, 0.0])    # centers, radius 1
4
5 def P_disk(x, c, r=1.0):
6     d = x - c
7     n = np.linalg.norm(d)
8     return c + d / n * r if n > r else x
9
10 def dr_step(x, gamma=1.0, lam=1.0):
11     y = P_disk(x, c2)           #  $J_{\{\gamma A2\}}$ 
12     z = P_disk(2 * y - x, c1)   #  $J_{\{\gamma A1\}}(2y - x)$ 
13     return x + lam * (z - y)
14
15 x = np.array([2.0, 2.0])
16 for n in range(4):
17     y = P_disk(x, c2)
18     print(n, "x=", x, " feasible pt y=", y)
19     x = dr_step(x)
20 # -> y converges to (0.851, 0.526) in C1 cap C2
```

4.3.3 Davis–Yin splitting – why, and the operator I

Problem 4.1.3 ($m = 2$, with a cocoercive piece back)

Find $x \in \mathcal{H}$ such that $0 \in (A_1 + A_2 + B)(x)$: A_1, A_2 maximally monotone, B β -cocoercive, $\text{zer}(A_1 + A_2 + B) \neq \emptyset$. (4.12)

Why would you want this?

Real objectives often have *three* pieces: two nonsmooth regularizers/constraints (A_1, A_2) plus one smooth data-fitting term (B). Neither FBS (built for one nonsmooth + one smooth piece) nor Douglas–Rachford (built for two nonsmooth pieces, no smooth part) directly applies. Davis–Yin splitting handles all three simultaneously, and specializes to FBS ($A_2 = 0$) or to Douglas–Rachford ($B = 0$).

4.3.3 Davis–Yin splitting – why, and the operator II

Lemma 4.4 – the averaged operator

Let $\gamma \in (0, 2\beta)$ and define

$$T_{DY} := J_{\gamma A_1} (2J_{\gamma A_2} - \text{Id} - \gamma B J_{\gamma A_2}) + \text{Id} - J_{\gamma A_2}. \quad (4.13)$$

Then T_{DY} is $\frac{2\beta}{4\beta - \gamma}$ -averaged, and $\text{zer}(A_1 + A_2 + B) = J_{\gamma A_2}(\text{Fix}(T_{DY}))$.

4.3.3 Davis–Yin splitting – why, and the operator III

The Davis–Yin splitting method

Let $\gamma \in (0, 2\beta)$, $\{\lambda_n\} \subset [0, \frac{4\beta-\gamma}{2\beta}]$ with $\sum_n \lambda_n (\frac{4\beta-\gamma}{2\beta} - \lambda_n) = \infty$. Choose $x_0 \in \mathcal{H}$; for each $n \geq 0$,

$$\begin{cases} y_n = J_{\gamma A_2} x_n, \\ z_n = J_{\gamma A_1} (2y_n - x_n - \gamma B y_n), \\ x_{n+1} = x_n + \lambda_n (z_n - y_n), \end{cases} \quad (4.14)$$

exactly the KM iteration $x_{n+1} = (1 - \lambda_n)x_n + \lambda_n T_{DY}(x_n)$. (4.15)

Spot the pattern

(4.14) is (4.10) with one extra term $-\gamma B y_n$ inserted into the middle line. Setting $A_2 \equiv 0$ (so $J_{\gamma A_2} = \text{Id}$, $y_n = x_n$) collapses (4.14) to FBS (4.7); setting $B \equiv 0$ collapses it to Douglas–Rachford (4.10). Davis–Yin genuinely *is* the common generalization.

4.3.3 Running example: adding a smooth term to two-disk feasibility

Instantiation

Same C_1, C_2 as before ($A_1 = N_{C_1}, A_2 = N_{C_2}$), now with $B = \text{Id}$ (gradient of $f(x) = \frac{1}{2}\|x\|^2$, 1-cocoercive). This solves

$$\min_{x \in C_1 \cap C_2} \frac{1}{2}\|x\|^2 \iff 0 \in (N_{C_1} + N_{C_2} + \text{Id})(x) :$$

the *minimum-norm* point of the feasible lens $C_1 \cap C_2$, not just *some* feasible point.

Numbers ($\gamma = 1, \lambda_n \equiv 1, x_0 = (2, 2)$)

n	$y_n = J_{\gamma A_2}(x_n)$
0	(1.447, 0.894)
1	(0.106, 0.211)
≥ 2	(0.000, 0.000)

Since the origin lies *on* ∂C_2 and at the center of C_1 , it is feasible and clearly of minimal norm – Davis–Yin finds it exactly, whereas plain Douglas–Rachford above found the different feasible point (0.851, 0.526): the extra cocoercive term B changes *which* point of $C_1 \cap C_2$ we converge to.

Python: Davis–Yin splitting

```
1 import numpy as np
2
3 c1, c2 = np.array([0.0, 0.0]), np.array([1.0, 0.0])
4
5 def P_disk(x, c, r=1.0):
6     d = x - c
7     n = np.linalg.norm(d)
8     return c + d / n * r if n > r else x
9
10 B = lambda x: x                # grad of  $f(x) = 1/2 \|x\|^2$ , 1-cocoercive
11
12 def dy_step(x, gamma=1.0, lam=1.0):
13     y = P_disk(x, c2)           #  $J_{\{\gamma A_2\}}$ 
14     z = P_disk(2 * y - x - gamma * B(y), c1) #  $J_{\{\gamma A_1\}}(\dots)$ 
15     return x + lam * (z - y)
16
17 x = np.array([2.0, 2.0])
18 for n in range(4):
19     y = P_disk(x, c2)
20     print(n, "min-norm feasible pt y=", y)
21     x = dy_step(x)
22 # -> y converges to (0, 0): the minimum-norm point of  $C_1 \cap C_2$ 
```

4.3.4 Primal-dual splitting – why, and the problem I

Why would you want this?

So far every splitting method needed *direct access* to each operator. But composite problems often involve a linear map, e.g. minimizing $f(x) + g(Lx)$ for a bounded linear L (think: total-variation regularization, constrained optimization $Lx = b$). Applying a resolvent to “ $g \circ L$ ” directly is typically intractable even when prox_g is easy. Primal-dual splitting handles A, B, C, D, L *all separately*, paying for L and L^* (its adjoint) only through matrix-vector products.

4.3.4 Primal-dual splitting – why, and the problem II

Problem 4.1.4

Let $\mathcal{H}, \mathcal{G}$ be real Hilbert spaces with (a) A maximally monotone, B β_B -cocoercive on $\mathcal{H}$; (b) C maximally monotone, D β_D -strongly monotone on $\mathcal{H}$; (c) $L : \mathcal{H} \rightarrow \mathcal{G}$ bounded linear. The *primal monotone inclusion* is

$$\text{Find } x \in \mathcal{H} : 0 \in (A + B)(x) + L^*((C \square D)(Lx)), \quad (4.16)$$

where $C \square D := (C^{-1} + D^{-1})^{-1}$ is the *parallel sum*. The *dual problem* is

$$\text{Find } v \in \mathcal{G} : (\exists x \in \mathcal{H}) \begin{cases} 0 \in (A + B)(x) + L^*v, \\ 0 \in (C^{-1} + D^{-1})(v) - Lx. \end{cases} \quad (4.17)$$

Assume both solution sets $\mathcal{P}$ (primal), $\mathcal{D}$ (dual) are nonempty.

4.3.4 Primal-dual splitting – why, and the problem III

(A) The primal-dual splitting iteration

Choose $\gamma_A, \gamma_C > 0$ with

$$2 \min\{\beta_B, \beta_D\} \min\left\{\frac{1}{\gamma_A}, \frac{1}{\gamma_C}\right\} \left(1 - \sqrt{\gamma_A \gamma_C \|L\|^2}\right) > 1. \quad (4.18)$$

Choose $x_0 \in \mathcal{H}$, $v_0 \in \mathcal{G}$; for each $n \geq 0$,

$$\begin{cases} x_{n+1} = J_{\gamma_A A}(x_n - \gamma_A B(x_n) - \gamma_A L^* v_n), \\ \bar{x}_{n+1} = 2x_{n+1} - x_n, \\ v_{n+1} = J_{\gamma_C C^{-1}}(v_n - \gamma_C D^{-1}(v_n) + \gamma_C L \bar{x}_{n+1}). \end{cases} \quad (4.19)$$

Reading it: a forward-backward step in x (treating $L^* v_n$ as a fixed forcing term), an over-relaxation $\bar{x}_{n+1}$, then a forward-backward step in v using the fresh $\bar{x}_{n+1}$ – primal and dual variables updated in an alternating, coupled fashion.

4.3.4 Primal-dual splitting – why, and the problem IV

(B) The fixed point reformulation

Stack primal and dual variables $z_n := \begin{pmatrix} x_n \\ v_n \end{pmatrix}$ in the product space $\mathcal{K} = \mathcal{H} \times \mathcal{G}$, and define block operators

$$\mathbf{A} := \begin{bmatrix} A & L^* \\ -L & C^{-1} \end{bmatrix}, \quad \mathbf{B} := \begin{bmatrix} B & 0 \\ 0 & D^{-1} \end{bmatrix}, \quad \mathbf{V} := \begin{bmatrix} \text{Id}_{\mathcal{H}}/\gamma_A & -L^* \\ -L & \text{Id}_{\mathcal{G}}/\gamma_C \end{bmatrix}. \quad (4.21)$$

Then (4.19) is equivalent to $-\mathbf{B}(z_n) \in \mathbf{A}(z_{n+1}) + \mathbf{V}(z_{n+1} - z_n)$ (Eq. 4.20), i.e.

$$z_{n+1} = (\mathbf{V} + \mathbf{A})^{-1}(\mathbf{V} - \mathbf{B})(z_n) = (\text{Id} + \mathbf{V}^{-1}\mathbf{A})^{-1}(\text{Id} - \mathbf{V}^{-1}\mathbf{B})(z_n). \quad (4.22)$$

4.3.4 Primal-dual splitting – why, and the problem V

The punchline

$\mathbf{A}$ is maximally monotone, $\mathbf{V}^{-1}\mathbf{B}$ is $\eta\rho$ -cocoercive with $\eta = \min\{\beta_B, \beta_D\}$ and $\mathbf{V}$'s positivity constant ρ set exactly by condition (4.18). So (4.22) is *forward-backward splitting*, Section 4.3.1's T_{FB} , run inside the Hilbert space $\mathcal{K}_{\mathbf{V}}$ induced by the self-adjoint positive-definite $\mathbf{V}$ (i.e. with inner product $\langle \cdot, \cdot \rangle_{\mathbf{V}}$ instead of the usual one). The fixed-point operator $\mathbf{T}_{FB} = J_{\mathbf{V}^{-1}\mathbf{A}}(\text{Id} - \mathbf{V}^{-1}\mathbf{B})$ is $\frac{2\eta\rho}{4\eta\rho - 1}$ -averaged: same averaged-operator template, just in a re-normed product space.

Summary: the unifying table

Method	Operator T	Averaging	Fix(T)
Gradient descent	$\text{Id} - \frac{2}{L} \nabla f$	nonexpansive	Argmin f
Proximal point	$J_{\gamma A}$	$\frac{1}{2}$ (firmly n.e.)	zer(A)
Forward-backward	$J_{\gamma A}(\text{Id} - \gamma B)$	$\frac{2\beta}{4\beta - \gamma}$	$J_{\gamma A}^{-1}$ -related to zer($A + B$)
Douglas–Rachford	$\frac{1}{2}(R_{\gamma A_1} R_{\gamma A_2} + \text{Id})$	$\frac{1}{2}$ (firmly n.e.)	maps via $J_{\gamma A_2}$ to zer($A_1 + A_2$)
Davis–Yin	T_{DY} , Eq. (4.13)	$\frac{2\beta}{4\beta - \gamma}$	maps via $J_{\gamma A_2}$ to zer($A_1 + A_2 + B$)
Primal-dual	FBS's T_{FB} in $\mathcal{K}_{\mathbf{V}}$	$\frac{2\eta\rho}{4\eta\rho - 1}$	product-space solu- tion (x^*, v^*)

Every row: $x_{n+1} = (1 - \lambda_n)x_n + \lambda_n T x_n$

Different T , same convergence machinery from Chapter 3. This is exactly why the KM iteration – with its built-in relaxation parameters λ_n – is the natural language for *designing* operator splitting methods, not just analyzing them after the fact.

Takeaways & what's next

- Gradient descent, the proximal point algorithm, forward-/backward-forward splitting, Douglas–Rachford, Davis–Yin, and primal-dual splitting all reduce to KM iteration $x_{n+1} = (1 - \lambda_n)x_n + \lambda_n T x_n$ for a suitable averaged T .
- The running example (projection onto the unit disk) threaded through every method: gradient descent (unconstrained quadratic), proximal point (single resolvent $= P_C$), forward-backward (quadratic + indicator), and its two-disk extension for Douglas–Rachford and Davis–Yin.
- The recurring design trick: split a hard joint resolvent $J_{\gamma(A+B+\dots)}$ into cheap pieces ($J_{\gamma A_i}$, gradient steps), then package the resulting map as one averaged operator so that KM theory applies verbatim.
- **Next (Chapter 5):** accelerating the KM iteration itself via inertial (heavy-ball-type) extrapolation terms – squeezing faster convergence out of the very same averaged-operator framework.